Thetawave.ai

💻 物件導向分析設計核心與模式運用

Generated by Thetawave.ai

ood-sop.png

🧩 物件導向設計 (Object-Oriented Design, OOD) 之思路與流程

> 「先讓程式能跑，再透過設計原則與設計模式精煉品質與彈性。」

主要原則與流程

階段	目標	方法與說明
起點	**先求跑**	使用**循序圖**(Sequence Diagram)梳理控制流程，確保程式能順利運作。
精煉	**再求好**	解決流程中衝突，提升程式品質，進而優化類別圖。
類別圖精煉	讓領域模型符合基本**設計原則**	透過最小知識原則等，提升彈性和維護性，持續優化模型。

> **循序圖**用於描繪物件互動流程，像是「故事腳本」，幫助理解系統行為。

📐 軟體架構設計方針與策略

大原則

- **當需求變化時，只修改最少量的類別與模組**，降低變更成本。

策略重點

策略	說明

封裝變化	將會一起變動的元素放在同一模組，不同變法則分開。
單向依賴	依賴方向由低階（衍生）模組指向高階（核心）模組。

實踐思路

- **模組化**：將變化方式與頻率相似的類別聚合成模組。
- **封裝三大關注點**：
  1. 以最剛好的可見度降低耦合
  2. 降低關係強度（解耦）
  3. 萃取剛好抽象，減少 Client 面對的資訊量

> **解耦**可比喻成「減少朋友間傳話的層級」，避免「朋友的朋友」產生不必要的聯繫。

🏷️ 類別設計的實踐細節

類別封裝原則

項目	說明
屬性私有化	所有屬性一律私有(private)，狀態只透過查詢操作提供。
去除雙向關聯	不允許兩個物件互相持有對方的參考，避免不必要耦合。
組合取代繼承	優先使用組合(composition)來實現功能，減少繼承帶來的複雜性。
區分主、被動物件	主動物件控制流程，被動物件提供服務，責任清晰。

依賴反轉原則(Dependency Inversion Principle, DIP)之重構三步驟

1. **表示職責/概念**：將變化封裝在特定類別中。
2. **萃取抽象**：將共用介面或抽象類別抽取出來。
3. **依賴抽象**：高階模組依賴抽象，低階模組實作抽象。

Thetawave.ai

🔍 設計模式應用觀察與分類

設計模式的思維流程

Christopher Alexander 模式思維

- 透過察覺(察覺 Forces)與定位，分類問題與解決方案。
- 以「衝突 (Problem)」、「解方 (Solution)」、「結果 (Resulting Context)」三步驟理解模式。

按設計意圖分類主要模式

分類	主要模式與特點
行為型模式	觀察行為變化，針對行為變動性進行設計。 - 觀察者模式 (Observer) - 策略模式 (Strategy) - 狀態模式 (State) - 指令模式 (Command)
創建型模式	高效且可擴充的物件創建方式。 - 單體模式 (Singleton) - 工廠方法 (Factory Method) - 抽象工廠 (Abstract Factory)
結構型模式	解耦 Client 與複雜結構，控制存取與相容性。 - 代理人模式 (Proxy) - 門面模式 (Facade) - 裝飾者模式 (Decorator) - 複合模式 (Composite) - 接器模式 (Adapter)
責任鏈模式	責任傳遞與處理流程中的動態擴充。 - 責任鏈模式 (Chain of Responsibility, CoR)

> 不同模式皆聚焦於**擴充性**與**變動性**的管理，透過模式達成高內聚低耦合。

🌀 行為型模式重點說明

行為型模式專注於物件之間的互動與行為變化，具代表性的幾種模式：

Thetawave.ai

模式名稱	主要功能說明
觀察者模式	當一個物件狀態改變時，通知其他依賴物件更新。
策略模式	定義一系列演算法，讓演算法可互換，且獨立於使用者。
狀態模式	根據內部狀態改變行為，避免大量條件判斷。
指令模式	封裝請求成物件，支持請求的排程、撤銷等操作。

這些模式的核心是**行為變動性**的管理，避免直接硬編碼的條件判斷，提升系統彈性。

🏭 創建型模式核心概念

創建型模式的目的是**高效率且高擴充性地管理物件創建**，避免直接使用 new 關鍵字造成的耦合。

模式名稱	特點
單體模式	保證類別只有一個實體，適用於系統中唯一資源管理。
工廠方法	定義介面由子類別決定要實例化的類別，支援擴充。
抽象工廠	提供介面創建一族相關或相依物件，確保產品系列一致性。

🧱 結構型模式重點

解決複雜結構耦合問題，提供更佳的重用性與擴充性。

模式名稱	功能說明
代理人模式	控制對目標物件的訪問，常用於權限、延遲加載等。
門面模式	提供簡單介面封裝複雜子系統，降低使用者學習成本。

裝飾者模式	動態地給物件增加新功能，替代繼承帶來的僵硬性。
複合模式	將物件組成樹狀結構以表示「部分-整體」層次，統一對待複合物件與單一物件。
接器模式	讓原本介面不相容的類別可以合作。

Mindwave.ai

💡 類別命名與概念萃取

- 類別應封裝**特定意義的詞彙**，形成清晰的**領域模型**。
- 以**最低階(Level 3)**的抽象開始，逐步向上精煉。
- 透過「點的萃取」，聚焦關鍵職責與概念，避免物件資料與行為過度散亂。

重要設計原則總結

原則名稱	核心說明
最小知識原則 (Least Knowledge Principle)	物件對其他物件只了解必要的資訊，避免「朋友的朋友」傳話。
封裝變化	將會一起改變的元素聚合，分離變化不同的部分。
單向依賴	依賴方向單向，避免循環依賴。
依賴反轉原則	高階模組不依賴低階模組，兩者共依賴抽象。
私有屬性與公開介面	屬性私有，僅提供剛好足夠的查詢與操作方法。
組合優先於繼承	使用組合來擴充功能，減少繼承帶來的複雜性與耦合。

設計工作流程建議

1. **先從主要流程出發**，透過循序圖掌握完整控制流程。

2. **初步完成類別圖**，確定核心類別與關係。
3. **逐步精煉類別圖**，依設計原則重構，降低耦合，提升彈性。
4. **察覺並分類 Forces（設計衝突與需求變動）**，選擇合適的設計模式套用。
5. **持續重構與優化**，實現高品質、易維護的軟體架構。

Thetawave.ai

設計模式與設計原則的整合，能讓軟體架構更具彈性與可維護性，並有效管理變動性與複雜度。

unnamed.png

🧩 物件導向分析(OOA)的詳細思路與需求萃取

> **物件導向分析(OOA)** 是將需求中隱含的概念和知識，針對職責與變化進行歸納與封裝，並以「類別」作為系統設計的基礎單位。此階段重點不在設計實作，而是**知識的抽取與組織**。

---

1. 需求萃取的核心原則

- **避免急於設計**：在分析需求時，切忌立刻跳向系統設計或程式實作階段。必須先聚焦在從需求中識別出所有相關的知識點和職責。
- **封裝職責與變化**：將需求中的職責（責任）與可能的變化歸納為類別，這有助於後續的系統擴展與維護。
- **明確分離知識與設計**：需求分析階段專注於「知識萃取」，設計階段才著手於類別結構與互動。

---

2. 需求文本的拆解方法

- **逐句分析**：將需求文本拆解成獨立的句子，以便逐句萃取概念。

Thetawave.ai

- **詞性識別**：對句子中的詞彙進行詞性標記，例如：
  - 名詞（可能代表類別或物件）
  - 動詞（可能代表行為或職責）
  - 形容詞（描述屬性或狀態）
- **關鍵詞萃取**：特別關注名詞與動詞的搭配關係，因為它們直接反映了系統中物件的狀態與行為。

---

3. 將詞彙轉化為類別與職責

類別/物件 (名詞)	職責/行為 (動詞)	說明
需求中出現的名詞	對應動詞	名詞代表系統中的實體或概念，動詞代表該實體的職責或操作
形容詞	-	描述類別的屬性或狀態，作為屬性設計的依據

> 例如：在「訂單管理系統」中，「訂單」是名詞，可能具有「建立」、「修改」、「取消」等動詞對應的職責。

---

4. 物件導向分析的思考比喻

可以將物件導向分析想像成「整理一張地圖」：

- **地圖上的地點**：對應系統中的類別（名詞）
- **地點之間的道路**：對應類別間的關係和互動（動詞）
- **地點的標誌與特徵**：對應類別的屬性（形容詞）

在這趟旅程中，分析師需要先認識每個地點的特性與功能，再理解它們如何連結，而非直接建造道路或建築物（設計與實作）。

Thetawave.ai

5. 易混淆重點解析

容易混淆點	解釋與建議
需求分析 vs. 系統設計	需求分析專注於抽取與整理知識；系統設計才是規劃類別結構與協作。
名詞與類別的區別	並非所有名詞都適合成為類別；需判斷其是否承載職責與狀態，能否封裝變化。
動詞與職責的辨認	動詞不只是行為，還代表類別的責任範圍，需結合語境判斷。
過早設計	過早設計會導致抓漏需求或錯誤封裝，影響系統彈性與可維護性。

---

6. 實務例子示範

假設需求句為：

> 「使用者可以建立訂單，並且修改訂單內容。」

名詞（類別候選）	動詞（職責）	屬性（形容詞/其他）	封裝說明
使用者	建立、修改	-	使用者負責操作訂單的建立與修改
訂單	建立、修改	內容	訂單是系統的核心資料，封裝屬性與操作

---

7. 物件導向分析的流程總結

步驟	重點說明
需求文本拆解	按句拆解，標記詞性
名詞與動詞萃取	識別類別候選與職責
知識封裝與歸納	將職責與變化封裝到類別中
避免設計干擾分析	不急於設計細節，聚焦於知識抽取

物件導向分析透過對需求中職責與變化的封裝，形成穩固且彈性的類別架構基礎，這有助於後續系統的擴展與維護。將需求拆解成名詞與動詞的配對，能清楚展現系統中各物件的角色與功能，避免漏失重要知識與誤入設計細節。這種分階段、分層次的思考方式是軟體架構設計的關鍵所在。

apply-behavioral-design-pattern-sop.png

行為變化型設計模式的分類與應用 SOP 🚦

> 在軟體設計中，**行為變動性**（Behavioral Variability）是指系統中某些操作的執行行為會因條件、狀態或輸入而改變。當察覺這種「行為變動力（Force）」時，設計者需要依據行為變化的特性分類，再判斷是否存在擴充性的需求，進而選擇合適的設計模式。

---

1. 行為變動性（Force-BV）的主要分類與特徵

分類名稱	特徵說明	舉例說明
原始型（Force-BV 原始型）	操作執行時，能在多種行為中選擇一種，且未來可持續擴充新的行為種類。	系統有多種處理方式，且會持續增加新方式，如多種付款方法的切換。
輸入比對型（Force-BV 輸入比對型）	系統先解析輸入參數的類型，根據不同類型執行對應處理，且未來可擴充新的輸入類型及其處理行為。	根據不同格式的輸入文件選擇相應的解析器，如 XML、JSON 或 CSV。
響應式（Force-BV 響應式）	當某類別完成特定行為並產生事件，其他多個類別需要根據該事件執行相應行為，且未來可擴充需響應此事件的類別。	類別 A 完成訂單後，類別 B 和 C 會分別執行通知客戶和更新庫存。
狀態型（Force-BV 狀態型）	某類別內多個屬性組合成不同狀態，依狀態不同，操作行為也不同，狀態越多，行為變化越強。	角色在「正常」和「中毒」狀態下攻擊方式不同，且有多種狀態組合。
行為賦予（Force-行為賦予）	某類別多個操作尚未定義行為，期望動態設定操作行為，透過呼叫其他類別的操作實現，行為可動態調整。	設定操作 A 執行時，實際呼叫其他類別的某個方法，類似動態指派行為。

N+1變化（Force-BV N+1變化）	操作分兩階段執行，階段一可任意組合多個「配菜」行為，階段二從「主菜」行為中選擇一種執行。	製作餐點時，先任意組合配菜，最後選定一道主菜完成。

Mindwave.ai

---

2. 擴充性的判定與對應設計模式

- **擴充性需求判斷：**
  當系統中的行為變化不僅存在，且未來必須方便地加入新行為或變更現有行為時，視為存在擴充性需求。
- **對應設計模式與推薦使用情境：**

擴充性需求存在時	對應設計模式	適用說明
複數行為可替換擴充	**策略模式**（Strategy Pattern）	將行為封裝成可互換的策略物件，透過多態性選擇執行。
多個處理者串聯處理	**責任鏈模式**（Chain of Responsibility Pattern）	將處理請求的責任沿著鏈條傳遞，符合多階段、順序處理需求。
事件發生多方響應	**觀察者模式**（Observer Pattern）	一對多依賴關係，事件發生時通知所有訂閱者，適合響應式行為變化。
狀態影響行為	**狀態模式**（State Pattern）	將狀態封裝成物件，根據狀態改變行為，避免大量條件判斷。
行為操作封裝並延遲執行	**指令模式**（Command Pattern）	將操作封裝為物件，可動態配置、排隊或撤銷。
行為組合導致爆炸性複雜	**裝飾者模式**（Decorator Pattern）	透過包裝原物件動態組合行為，避免繼承過多子類。

---

3. 具體應用比喻與說明

- **原始型：**
  就像點餐時你能在「漢堡」、「薯條」、「飲料」中選一樣，而且未來菜單還會新增其他種類。

Thetawave.ai

- **輸入比對型：**
  如同郵件系統根據郵件類型（普通郵件、垃圾郵件、重要郵件）做不同處理，且可擴增新類型。
- **響應式：**
  比喻為社交媒體中，當某人發表貼文（事件），好友們會收到通知並作出反應。
- **狀態型：**
  想像遊戲角色因為「正常」、「中毒」、「暈眩」不同狀態，攻擊與防禦行為不同。
- **行為賦予：**
  就像遙控器按鈕尚未設定功能，可以動態配置按下按鈕會啟動何種設備或動作。
- **N+1變化：**
  類似做一道套餐，先選任意配菜（沙拉、湯、甜點等），最後選一道主菜完成整體餐點。

---

4. 設計模式選擇流程（SOP）

1. **識別行為變動力（Force）類型**
   透過關鍵句型自下而上判定分類，明確是哪一種行為變動性。
2. **判斷是否存在擴充性需求**
   是否需要未來擴充行為或動態調整行為。
3. **根據分類與擴充性選擇設計模式**
   按照表格對應關係，選用策略、責任鏈、觀察者、狀態、指令或裝飾者模式。

---

5. 重點整理

- **行為變動性是軟體設計中常見且關鍵的挑戰**，分類清楚有助於選擇合適模式。
- **擴充性是選擇設計模式的主要動因**，沒有擴充需求時可能不需要複雜模式。
- **每種行為變動型態對應不同設計模式**，避免誤用導致複雜度提高。
- **比喻有助理解**，將抽象行為變化對應到生活場景易於記憶與應用。
- **設計模式的目標是減少條件判斷，提升系統彈性與可維護性。**

行為變動型設計模式的分類與擴充性需求緊密相連，選擇合適的結構不僅能有效管理行為變化，也能促進系統的可擴展性與維護便利。系統中多態性與事件驅動的設計相輔相成，使變化管理更具彈性且不破壞原有架構。

full-sop.png

🧩 軟體設計模式精通之旅：思路全貌 SOP

> 「要空杯，先做慢、做慢、做精，再做快。」
> 這句話強調學習設計模式的心態與步驟：先放下既有成見，從基礎開始慢慢打好根基，再逐步提升效率。

---

1. 起點：需求理解的挑戰與設計約束

當發現自己對需求中的概念理解不完整，或遇到新的設計約束與難題時，必須回到分析與設計的根本，以確保後續設計的正確性與彈性。

---

2. 物件導向分析（Object-Oriented Analysis，OOA）

主要關注點

Thetawave.ai

- **捕捉需求中的概念與知識**
  將需求中的各項職責（Responsibility）與變化脈絡封裝至類別中，不急於設計實作細節。
- **萃取領域知識**
  從需求文本中辨識重要詞彙，區分「實體（Entity）」、「屬性（Attribute）」與「行為（Behavior）」。

重要區分

概念	說明	範例說明
實體（Entity）	可實際存在的物件	「雞」作為一個實體
屬性（Attribute）	實體的特徵或狀態	「青」只是描述「雞」的顏色屬性，不是實體
行為（Behavior）	實體能執行的動作或功能	「雞」能「下蛋」、「走路」等

> 這裡的比喻：**「青」是雞身上的一種顏色，如同衣服的顏色，是屬性，不是獨立存在的物件。**

---

3. 物件導向設計（Object-Oriented Design，OOD）

設計流程

- **先求跑，再求好**
  先用循序圖（Sequence Diagram）梳理互動流程，確保程式能執行，然後再精準實踐設計原則與模式。
- **循序圖引導控制流程設計**
  從主要流程開始，完整設計物件間的互動。
- **初步精煉類別圖**
  透過設計原則使領域模型具備彈性與可維護性。

精煉目的

精煉方向	具體目標
解決衝突	讓設計有良好品質
實踐設計原則	增強彈性與維護性

Thetawave.ai

---

4. 基本設計原則與思路

最小知識原則（Least Knowledge Principle）

> 「物件對其他物件只有最少的了解。」
> 這原則強調物件之間的低耦合，避免不必要的依賴。

三大核心關注點

關注點	解釋	實踐方式
封裝（Encapsulation）	將資料和行為包裹起來，隱藏內部細節	控制物件的可見度與存取權限
解耦（Decoupling）	減少物件間依賴	使用介面、抽象類別
萃取（Extraction）	找出合適的抽象	提煉共通行為與屬性

---

5. 實踐思路：依類反轉的重構三步驟

- **依類反轉（Class Inversion）**指的是在重構過程中，根據發現的新需求或設計力（Force）重新調整類別結構。
- **無招勝有招**：先不急於套用固定模式，先發現需求中的新的力，再精準套用設計模式。

> 這裡借用Christopher Alexander的「模式思維」，強調根據「Force→Problem→Solution→Resulting Context」的流程，精準使用設計模式。

Thetawave.ai

6. 軟體架構設計的核心流程

流程步驟	內容說明
察覺 Forces	觀察需求中存在的衝突與挑戰
定義 Problem	明確問題本質
提出 Solution（Form）	選擇合適的模式或架構解決問題
形成 Resulting Context	新架構下無問題的狀態

---

7. 物件與類別的萃取

點的萃取

- 從需求中找出具體的**物件（Object）**。
- 判斷是否為實體（Entity）或僅屬於屬性（Attribute）等。

類別的萃取

- 若無法單純以實體區分，則根據共同特徵將實體內部元素分類，形成**類別（Class）**。

---

8. 線的萃取（關係分析）

需求中關係的辨識

關係類型	定義	代表關係	說明
一般化/繼承（Generalization/Inheritance）	表示「是...的一種」關係	IS-A	如「貓是動物的一種」
關聯（Association）	表示物件間的連結	HAS-A	如「學生有課程」
依賴（Dependency）	使用關係，存在於行為之外	使用	如方法中使用另一物件

關聯分析要點

- 以需求中的關鍵詞與句型定位參與者與數量（基數）。
- 升級關係成為聯類別（Association Class）以處理複雜互動。

Thetawave.ai

---

9. 需求詞彙分類示例表

詞彙類型	例子	解說
實體（Entity）	「雞」	具體存在的物件
屬性（Attribute）	「青」	實體的特性，非獨立物件
行為（Behavior）	「下蛋」	實體可執行的動作
關係（Association）	「雞與蛋的關係」	物件間的聯繫

---

10. 物件導向設計中常見的誤區與易混淆點

易混淆點	說明	建議理解方法
屬性與實體混淆	屬性是描述實體的狀態，不是獨立物件	把屬性想像成物件的「外衣顏色」或「身高」
依賴與關聯搞混	依賴是臨時性的使用，關聯是持續性的連結	依賴像「借用」，關聯像「朋友」
先求跑再求好	忘記先把流程跑通就急著優化	類比「先搭好舞台，再布置佈景」

---

11. 領域模型的結構與行為

領域模型是由**結構（類別、屬性、關係）**與**行為（物件方法）**組成。良好領域模型應：

- 清楚反映真實世界的概念與規則。
- 支援需求變更與擴充。
- 遵守設計原則，維持高內聚、低耦合。

Thetawave.ai

12. 循序圖在設計中的應用

- **循序圖**用於描述物件間的互動流程，特別是控制流程的設計。
- 從**主要流程開始，完整梳理控制流程**，確保系統的動態行為正確。
- 是「先求跑」的重要工具，確保程式運作正常。

---

13. 設計模式運用的思考框架

元素	定義	作用
Forces（力）	設計中的衝突與限制	理解問題本質
Problem（問題）	具體設計挑戰	找出解決目標
Solution（解決方案）	模式或架構設計	提供通用解決方式
Resulting Context（結果情境）	解決後的系統狀態	評估設計成效

物件導向分析與設計的流程中，從需求詞彙的細緻萃取，到循序圖導引的動態流程，再到初步精煉的類別結構，彼此間的關聯構成一個密不可分的系統。設計原則如最小知識原則促使物件間保持低耦合，而封裝與解耦則確保系統的彈性與可維護性。設計模式的應用須基於對需求中各種力的察覺，透過問題與解決方案的反覆迭代，形成具備彈性且易擴充的軟體架構。

ooa-sop.png

🧩 軟體設計模式精通之旅—物件導向分析（OOA）核心概念整理

> **物件導向分析（OOA, Object-Oriented Analysis）**是在需求階段透過捕捉需求中的概念與知識，將需求中的職責及變化脈絡封裝進類別，為後續設計奠定基礎。這一階段重點在於**知識萃取**，不急於設計細節。

Thetawave.ai

1. 需求到物件導向分析的轉換流程

步驟	說明
餵入需求	將需求中重要且具特定意義的概念提取出來。缺乏對這些概念的理解會影響需求實現。
詞彙是否為獨立實體（Entity）	判斷詞彙是否代表一個可實際存在且具意義的實體。
開出新物件（Object）	若詞彙為實體，則將其對應為系統中的物件。
開出新類別（Class）	為物件分類，建立類別。
開出新屬性（Attribute）	分析句子中對實體的相關描述，找出實體的屬性並附加到實體上。

2. 物件導向分析中的核心概念與區分

概念	定義與說明
實體 (Entity)	具重要意義且可實際存在的物件或類別。
職責 (Responsibility)	實體存在的意義，是對實體的高階描述或定義。
屬性 (Attribute)	描述實體特徵的一組值，格式通常為 name: type。
行為 (Behavior)	實體基於職責在系統中所執行的動作，必須用程式碼清楚實現。
操作/動作 (Operation)	行為的具體方法，與行為相對應。
互動 (Interaction)	實體間行為的協同，需要使用其他實體的能力。

3. 關係（Relationship）分析與分類

關係是根據實體定義或行為歸納出的連結，主要分為三種：

關係類型	說明	代表語義與分類

Is-A	表示「繼承」或「一般化」關係，屬於靜態分類層級。	繼承（Inheritance）、類別—般化
Has-A	表示「擁有」或「組合」關係，屬於靜態結構關聯。	關聯（Association）
Uses-A	表示動態的依賴關係，一方行為中需要另一方的能力，屬於動態依賴。	依賴（Dependency）

Newwave.ai

關係判斷流程

1. **是否存在於行為之外？**
   - 是 → 可能屬於靜態關係（Is-A 或 Has-A）
   - 否 → 屬於動態依賴（Uses-A）
2. **Is-A 還是 Has-A？**
   - 若表示分類或繼承 → Is-A
   - 若表示擁有或結合 → Has-A
3. **基數分析（Cardinality）**
   - 分析兩端實體間同時可關聯的數量（例如一對多、多對多）。

---

4. 具體分析實例說明

以需求中一句話為例，進行分析：

- **詞彙辨識：**
  將句中的關鍵詞判斷是否為實體。
  - 是實體 → 開出物件並分類成類別
  - 非實體但描述特徵 → 設為屬性
- **屬性開出：**
  針對實體的描述，找出屬性並附加。
  例：name: string, age: int

Thetawave.ai

- **關係判定：**
  從定義和行為中分析實體間的關係類型（Is-A、Has-A、Uses-A），並分析基數。

---

5. 實體與職責的關係

實體(Entity)	職責(Responsibility)	行為(Behavior)
具體的物件或類別	為何存在，是其高層定義與意義	根據職責設計的功能和操作，必須程式實作

---

6. 關係類型詳細說明與示意

關係類型	特徵說明	範例
Is-A	類別之間的繼承關係，代表一種「是……」的分類	「貓」是「動物」，「汽車」是「交通工具」
Has-A	類別擁有另一類別的成員，表示組合或聚合關係	「汽車」有「輪胎」，「房子」有「門窗」
Uses-A	動態依賴，行為中一方需要另一方的能力，非持續擁有	「司機」使用「汽車」來駕駛

---

7. 實體間基數（Cardinality）分析

基數用於描述關係中雙方在同一時間可同時存在的數量約束：

方向	問題範例	說明
A → B	每個 A 同時可關聯多少個 B？	如一個「老師」可教多個「學生」。
B → A	每個 B 同時可關聯多少個 A？	如一個「學生」可有多個「老師」。

Thetawave.ai

8. 物件導向分析的重點提醒

- **不要急於設計**，先專注於從需求中萃取出關鍵知識與職責。
- **分類清晰**，將實體、屬性、行為、職責、關係明確區分。
- **關係判斷**是理解系統架構的關鍵，要明確區分靜態與動態關係。
- **程式碼實現的基礎**是分析階段的行為定義，必須能轉換成具體操作。

物件導向分析階段將需求中的概念有系統地組織起來，形成清晰的物件模型。透過精準的實體辨識、職責分配、屬性定義及關係分類，能夠建立穩固的系統基礎，方便後續設計與開發環節。合理的分類與關係界定不僅使系統結構更清晰，也使職責分工更明確，提升軟體的可維護性和擴展性。